

Параллельные высокопроизводительные вычисления

Отчёт по заданию

**“многопоточная реализация операций с сеточными
данными на неструктурированной смешанной сетке,
решение СЛАУ”**

Вариант – Б1

Группа: 523

Повжик Юрий Максимович
23.11.2025

Содержание

2 Описание задания и программной реализации.	3
2.1 Краткое описание задания	3
2.2 Краткое описание программной реализации.....	4
3 Исследование производительности	5
3.1 Характеристики вычислительной системы	5
3.2 Результаты измерений производительности	7
3.2.1 Исследование роста потребления памяти.....	7
3.2.2 Сравнение MPI и OpenMP.....	8
3.2.3 Параллельное ускорение	9

2 Описание задания и программной реализации.

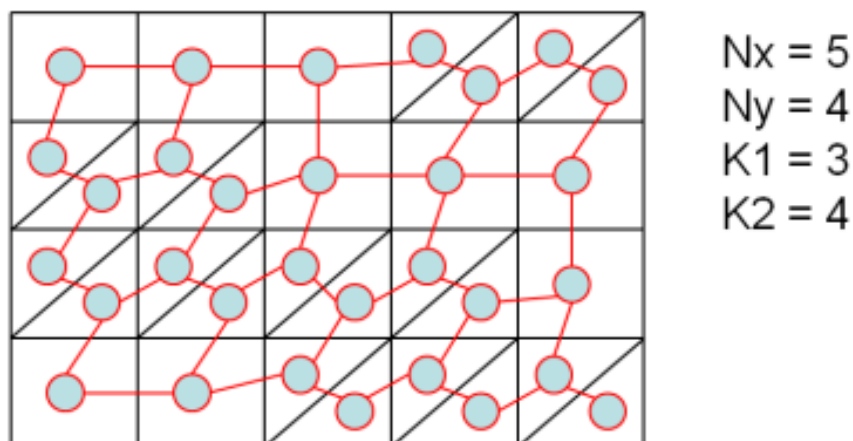
2.1 Краткое описание задания

Задание: многопоточная реализация операций с сеточными данными на неструктурированной смешанной сетке, решение СЛАУ.

Входными данными для задачи являются:

- N_x, N_y – число клеток в решетке по вертикали и горизонтали
- P_x, P_y – указывают, на сколько частей мы делим решетку по соответствующим направлениям. Общее число MPI процессов должно быть $P = P_x \times P_y$.
- K_1, K_2 – параметры для количества однопалубных и двухпалубных кораблей треугольных и четырехугольных элементов
- maxit - максимальное число итераций метода сопряженных градиентов с диагональным предобуславливателем
- eps - критерий остановки (ϵ), которым определяется точность решения
- Y - флаг для вывода отладочной печати

Согласно варианту задания был реализован следующий вариант интерпретации входных данных:



Для решения задачи необходимо реализовать следующие этапы:

1. Generate – генерация графа/портрета по тестовой сетке;
2. Fill – заполнение матрицы по заданному портрету;

3. Com – построение схемы обменов;
4. Solve – решение СЛАУ с полученной матрицей;
5. Report – проверка корректности программы и выдача измерений.

2.2 Краткое описание программной реализации

Для проверки и дальнейшего хранения входных параметров задачи был реализован класс **StartData**, принимающий argc и argv, rank и size.

В случае успешной проверки мы формируем стартовую матрицу, класс которой назван **StartMatrix**. Он принимает **StartData** и строит по полученным данным матрицу соответствующего вида. Элементы матрицы имеют тип **MatrixElem**, хранящий четыре значения: локальные и глобальные индексы верхнего и нижнего элементов. Метод **fillDistributedMatrix** – заполняет глобальные индексы внутренних элементов, **fillNeighbors** – глобальные индексы гало, **initLocalIndexes** – заполнение локальных индексов: сначала происходит нумерация внутренних элементов, затем гало: верх, лево, право, низ.

Построением формата ELLPACK занимается класс **ELLPACK**, принимающий на вход **StartMatrix** и **StartData**. Он хранит матрицу значений A, JA, массив LocalToGlobal. Метод **fillJA** заполняет матрицу JA. Места отсутствующих элементов заполняем последним встреченным значением. Методы **addPair** и **addElem** вспомогательные и используются для добавления ребер между точками. Метод **fill** заполняет матрицу A сгенерированными значениями.

Класс **Comm** отвечает за обмен данными между процессами. Он хранит в себе массивы, содержащие номера вершин, от которых мы данные получаем и номера вершин, данные которых отправляем.

Функция **fillB** заполняет массив b.

Также в коде присутствуют реализации функции **spmv** для диагональной матрицы и матрицы в формате ELLPACK, **dotVec**, **axpy**, **copyVec** и **fillConstVal**, которые принимают в себя помимо основных аргументов параметр для измерения времени.

В функции **generate** производится обработка входных параметров задачи и формирование **StartMatrix** и **ELLPACK**.

В **fill** – заполнение матрицы значений A из ELLPACK соответствующим методом и

заполнение вектора правой части b .

B solve – решение задачи методом сопряженных градиентов.

B result – вывод результатов измерений.

Для запуска программы необходимы 6 аргументов: N_x , N_y , P_x , P_y , $K1$, $K2$. N_x , N_y – размеры матрицы. P_x , P_y – число процессов по вертикали и горизонтали. $K1$, $K2$ – число подряд идущих однопалубных и двупалубных элементов. Седьмой параметр Y опциональный. При его вводе будут показаны результаты, полученные в задаче, такие как стартовая матрица, ее представление в формате ELLPACK, матрица A , вектор b и x .

Пример запуска программы для отладки:

```
mpirun -np 4 ./p2 4 4 2 2 2 1 Y
```

Число используемых процессов должно быть равно $P_x * P_y$, что проверяется в **StartData**.

3 Исследование производительности

3.1 Характеристики вычислительной системы

Запуск задания производился на Polus со следующими характеристиками:

Пиковая производительность	55.84 TFlop/s
Производительность (Linpack)	40.39 TFlop/s
Вычислительных узлов	5

На каждом узле:

Процессоры IBM Power 8	2
NVIDIA Tesla P100	2
Число процессорных ядер	20
Число потоков на ядро	8
Оперативная память	256 Гбайт

Коммуникационная сеть	Infiniband / 100 Gb
Система хранения данных	GPFS
Операционная система	Linux Red Hat 7.5

Память:

На одном узле — 2 сокета

На каждом сокете:

4 планок DDR-2400 по 32 GB (Всего 128 GB)

2 кентавра (L4 кэш + контроллер памяти)

Кентавр - 4 канала памяти

(8 каналов на сокет, 16 каналов на узел)

L1 кэш – 64KB

L2 кэш – 512KB

L3 кэш – 8 MB/ядро

L4 кэш – 16 MB на кентавр (32 MB на сокет)

Пиковая пропускная способность на сокете – 153.6 GB/s

Пиковая пропускная способность на узле – 307.2 GB/s

Компиляция программы производилась следующим образом:

```
mpicxx -o3 p2.cpp -o p2
```

```
g++ (GCC) 9.3.1 20200408 (Red Hat 9.3.1-2)
```

Пример запуска программы:

```
#!/bin/bash
#BSUB -J my_mpi_pr
#BSUB -n 16
#BSUB -x
#BSUB -R "span[ptile=8]"
#BSUB -m "polus-c3-ib polus-c4-ib"
#BSUB -W 0:30
#BSUB -o pr_4_16.%J.out
#BSUB -e pr_4_16.%J.err
```

```
echo "=====
echo "Running with MPI processes = 16"
echo "=====
```

```
mpirun --map-by core --report-bindings -np 16 ./pr 2500 3000 4 4 2 1
```

3.2 Результаты измерений производительности

3.2.1 Исследование роста потребления памяти

Для проверки производительности рекомендуется взять число элементов равное следующим величинам

- 1 000 000
- 10 000 000

Для начальных параметров N_x , N_y , K_1 и K_2 выбраны следующие значения:

- 750 1000 2 1
- 2500 3000 2 1

Параметры $\maxit$ и ϵ предлагается взять равными 1000 и 0.001 соответственно. В качестве R_x и R_y выбираются максимально близкие значения для каждого запуска. Например: для 4 – 2 и 2; для 8 – 2 и 4.

Проверка потребления памяти для 20 процессов:

1) Вход: 750 1000 2 1

- $N_{cells} = 750\,000$
- $size = 1\,000\,000$
- $matrix$ (всего) = 9 000 000 байт = 8.583 MiB
- A (всего) = 40 000 000 байт = 38.147 MiB
- JA (всего) = 20 000 000 байт = 19.073 MiB
- $currentPos$ (всего) = 4 000 000 байт = 3.815 MiB
- $Comm$ (всего) $\approx 586\,400$ байт = 0.559 MiB
- Пиковое (генерация, всего) $\approx 73\,586\,400$ байт = 70.178 MiB
- b (всего) = 8 000 000 байт = 7.629 MiB
- Пиковое (заполнение, всего) $\approx 68\,586\,400$ байт = 65.398 MiB

2) Вход: 2500 3000 2 1 2000 0.001

- $N_{cells} = 7\,500\,000$
- $size = 10\,000\,000$
- $matrix$ (всего) = 90 000 000 байт = 85.831 MiB
- A (всего) = 400 000 000 байт = 381.470 MiB
- JA (всего) = 200 000 000 байт = 190.735 MiB

- currentPos (всего) = 40 000 000 байт = 38.147 MiB
- Comm (всего) $\approx$ 1 813 600 байт = 1.724 MiB
- Пиковое (генерация, всего) $\approx$ 731 813 600 байт = 697.726 MiB
- b (всего) = 80 000 000 байт = 76.294 MiB
- Пиковое (заполнение, всего) $\approx$ 681 813 600 байт = 650.269 MiB

Таким образом мы имеем линейный рост потребления памяти.

3.2.2 Сравнение MPI и OpenMP

Ниже приведены таблицы ускорение трех базовых операций и всего метода Solve в зависимости от числа потоков для сетки размера 10_000_000 для OpenMP и MPI соответственно:

OpenMP					
Потоки	Solve	SPMV	SPMV_Diag	DOT	AXPY
4	3.820830	4.066774	4.056535	3.912584	3.344756
9	5.076676	4.946102	4.963137	7.166202	3.888777
16	8.487693	8.248979	8.278547	11.226265	7.214026
MPI					
Потоки	Solve	SPMV	SPMV_Diag	DOT	AXPY
4	3.769704	3.90963606	3.90699207	3.94366632	3.37254446
9	6.91121486	8.06555306	6.17773249	8.58289084	7.09876751
16	8.33921161	9.39747259	7.15094554	14.61735642	7.09876751

По результатам видно, что программы показывают близкие значения ускорения на большом числе потоков. При этом MPI быстрее приближается к своему максимальному ускорению.

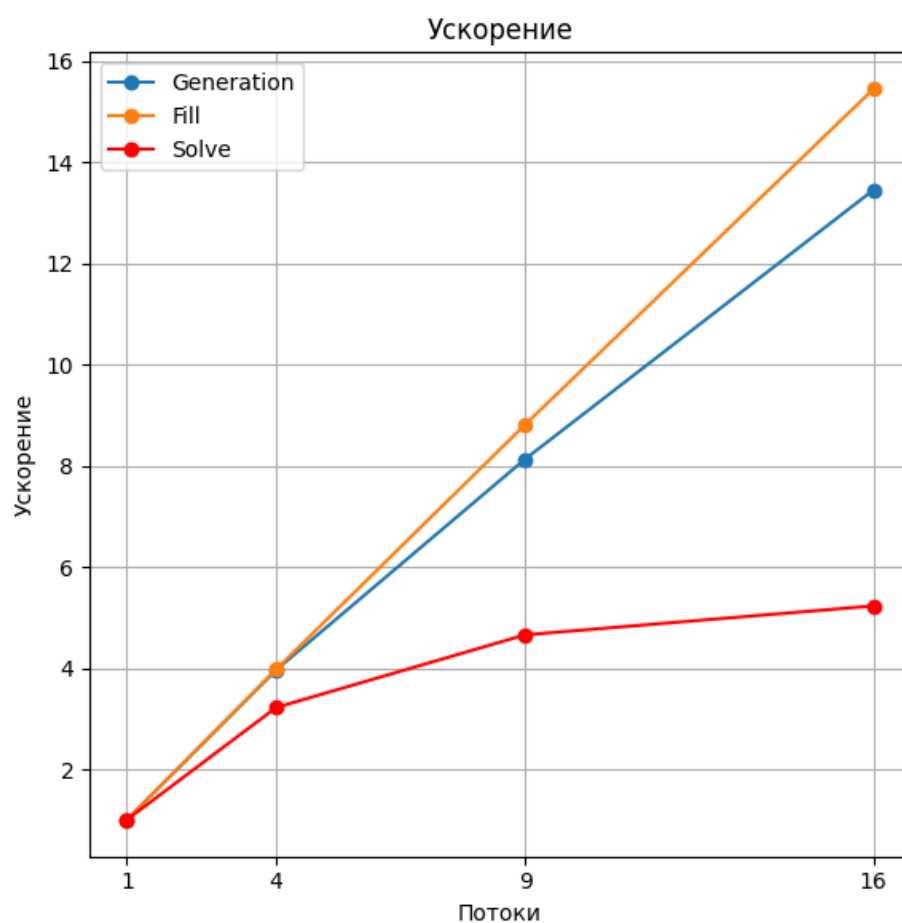
На 9 процессах более высокое ускорение показывает MPI за счет более равномерного распределения процессов по сокетам.

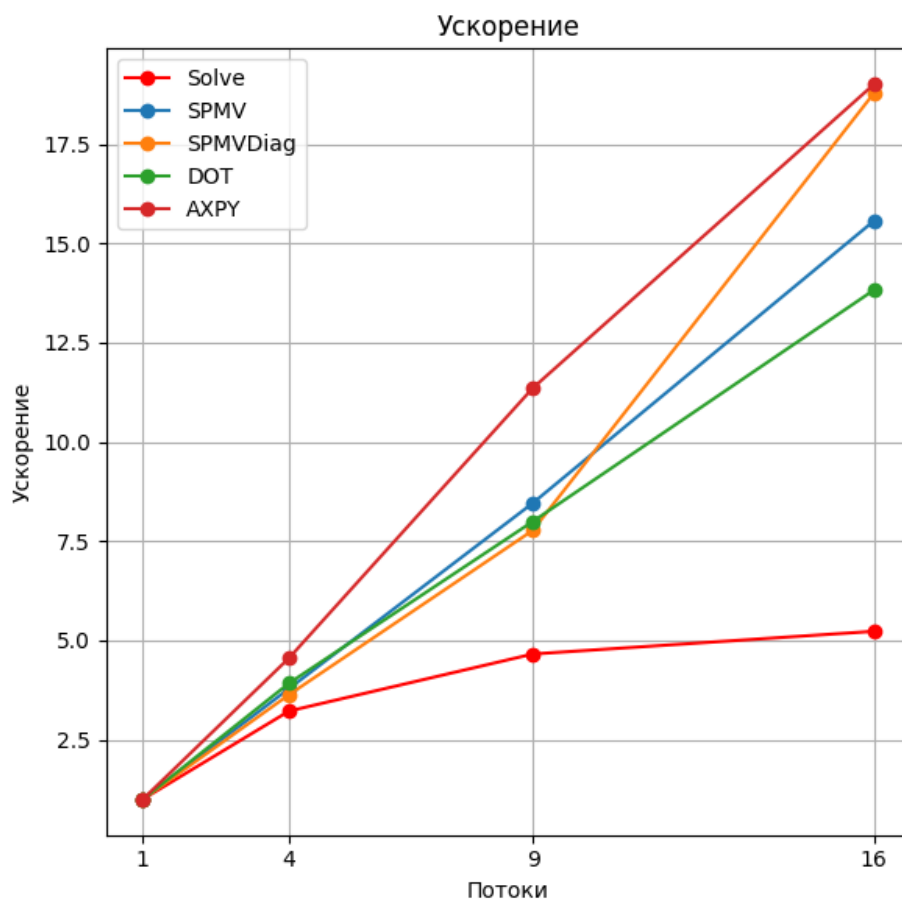
3.2.3 Параллельное ускорение

Ниже приведены таблицы времени выполнения и графики ускорений для каждого N:

1) N = 1 000 000

Число потоков	Generate	Fill	Solve	SPMV	SPMV_Diag	DOT	AXPY
1	0.0239902	0.252437	0.179857	0.00503372	0.00261897	0.00153399	0.00118975
4	0.00605341	0.0634968	0.0558435	0.00132963	0.000720872	0.000391073	0.000261098
9	0.00295075	0.028648	0.0386201	0.000595165	0.000337024	0.00019188	0.000104778
16	0.00178339	0.0163411	0.0343934	0.000323385	0.000139432	0.000110961	0.0000626

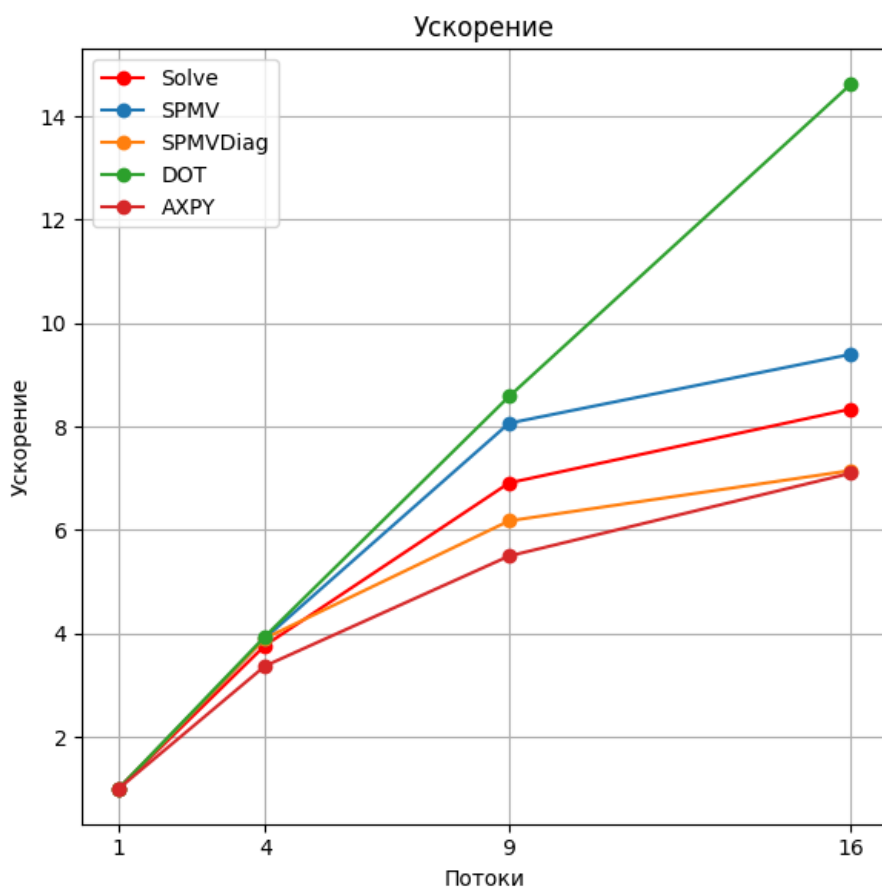
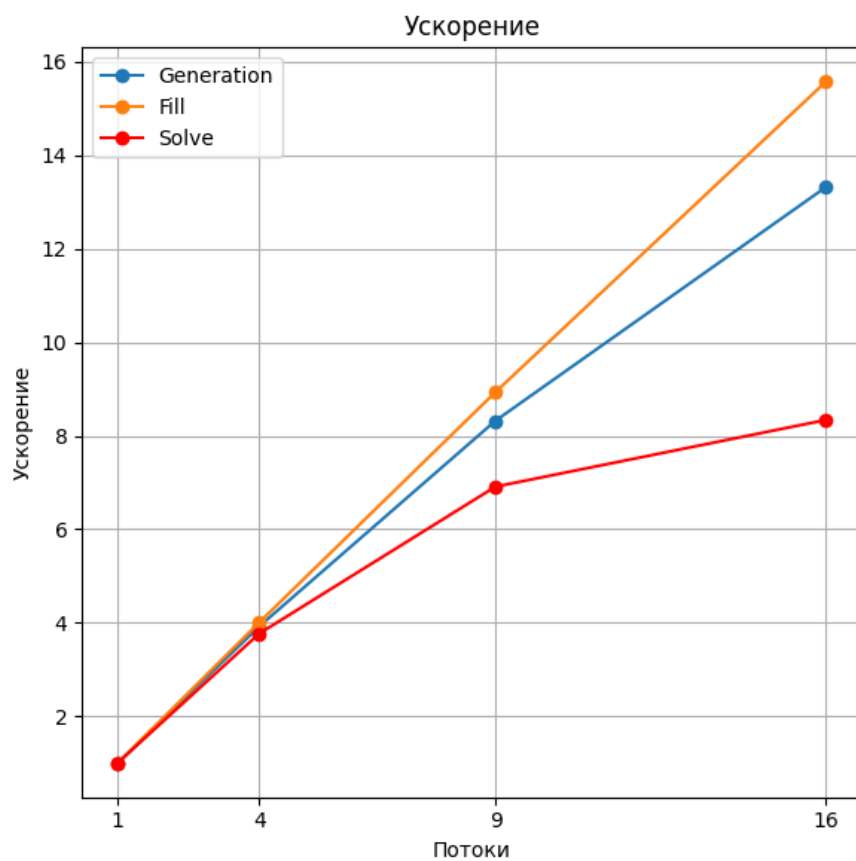




В виду малого размера матрицы мы получаем низкий прирост ускорения метода Solve из-за затрат на пересылку данных.

2) $N = 10\,000\,000$

Число потоков	Generate	Fill	Solve	SPMV	SPMV_Diag	DOT	AXPY
1	0.239399	2.53182	2.00739	0.0490284	0.0270408	0.0153396	0.0147851
4	0.0610349	0.631826	0.532506	0.0125404	0.00692113	0.00388968	0.00438396
9	0.028774	0.283456	0.290454	0.00607874	0.00437714	0.00178723	0.00268924
16	0.0179731	0.16256	0.240717	0.00521719	0.00378143	0.00104941	0.00208277



При $N = 10\,000\,000$ затраты на обмен данными метода Solve становятся менее существенными, что улучшает значения ускорения по сравнению с $N = 1\,000\,000$.